

Assignment 2: Coding Basics

Elizabeth Miles-Flynn

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1.  
seq(1,55,5) #Generates a sequence of numbers from 1 to 55, increasing by fives.
```

```
## [1] 1 6 11 16 21 26 31 36 41 46 51
```

```
#Resulting in the following: 1,6,11,16,21,26,31,36,41,46,51  
Increases_by_Five <-seq(1,55,5) #Assign a name to the sequence
```

```
#2.  
# Summary statistics for both mean and median, putting "Increases_by_Five" within the parentheses  
# so that we only calculate values from the sequence provided.  
mean(Increases_by_Five)
```

```
## [1] 26
```

```
mean_value <-mean(Increases_by_Five) #Assign a name to the mean value  
median(Increases_by_Five)
```

```
## [1] 26
```

```
median_value <- median(Increases_by_Five) #Assign a name to the median value  
  
#3.  
#use assigned names for both mean_value and median_value  
#so that we can create a conditional statement  
mean_value > median_value #when you run this line, R will provide a "false" conclusion
```

```
## [1] FALSE
```

```
#for the conditional statement because the mean  
#and median have the same value of 26.
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
# Vectors ----  
vector1 <- c("Sam","Lindsey","Belle","Charles") # Vector type: character  
vector1
```

```
## [1] "Sam"      "Lindsey" "Belle"   "Charles"
```

```
class(vector1)
```

```
## [1] "character"
```

```
vector2 <- c(96,87,79,84) # Vector type: numeric  
vector2
```

```
## [1] 96 87 79 84
```

```
class(vector2)
```

```
## [1] "numeric"
```

```
vector3 <- c(TRUE,TRUE,FALSE,FALSE) # Vector type: logical  
vector3
```

```
## [1] TRUE TRUE FALSE FALSE
```

```
class(vector3)
```

```
## [1] "logical"
```

```
#Create a data frame for each vector
```

```
df_vector1 <- as.data.frame(vector1)
```

```
df_vector2 <- as.data.frame(vector2)
```

```
df_vector3 <- as.data.frame(vector3)
```

```
#combining all vectors into a singular data frame, assigning an informative name
```

```
Academic_Performance_By_Scholarship_Status <-
```

```
  cbind(df_vector1,df_vector2,df_vector3)
```

```
class(Academic_Performance_By_Scholarship_Status)
```

```
## [1] "data.frame"
```

```
#Change labels for each column with informative names
```

```
colnames(Academic_Performance_By_Scholarship_Status) <-
```

```
  c("Student_Name","Test_Score","Own_Scholarship")
```

9. QUESTION: How is this data frame different from a matrix?

Answer: A data frame can store and combine data from a variety of different modes like character, numeric, integer, logical, and complex. You will see only one type of mode in each column in a data frame. This is different than a matrix where you can only have one type of mode for all values. For example, a matrix would have all numerical data.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word “Pass”; otherwise print the word “Fail”.
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else
```

```
pass_if <- function(Test_Score) {if(Test_Score>50) {"Pass"} else {"Fail"}}
```

```
#11. Create a function using ifelse()
```

```
pass_ifelse <- function(Test_Score) {ifelse(Test_Score>50,"Pass","Fail")}
```

```
#12a. Run the first function with the value 52.5
```

```
pass_if(52.5)
```

```
## [1] "Pass"
```

```
#12b. Run the second function with the value 52.5  
pass_ifelse(52.5)
```

```
## [1] "Pass"
```

```
#13a. Run the first function with the vector of test scores  
#pass_if(vector2)
```

```
#13b. Run the second function with the vector of test scores  
pass_ifelse(vector2)
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: `ifelse` worked for running the `vector2` values and outputting results to showcase whether or not the test scores are “passing” grades. Only `ifelse` is capable of applying the conditional operation “if” to an entire vector data, meaning that `ifelse` can run the condition statement for all values within vector 2 for Test Scores. `if...else` did not work because it can only test one value’s logical condition at a time. Since there are four components in the “Test_Scores” (`vector2`), the multiple values caused an “error” in R.

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)